fogp5dnnv

April 15, 2025

1 Shivali Homework 7 - Log Scale

In this homework, we will be using different datasets to explore the use of log scales in visualizations. Logarithmic scales are useful when dealing with data that spans several orders of magnitude and ratio-based comparisons. By the end of this assignment, you will create visualizations using log scales and interpret the results.

1.1 Instructions

1. **Project Setup**:
   - Set up your Python and Jupyter (or VSCode) environment.
   - Clone or download the repository provided in class (refer to the class notes).
2. **Fill the cells**:
   - Fill in the cells with the code provided in the instructions.
   - You can use the provided code as a starting point and modify it as needed.
   - Make sure to run the code in each cell to see the output.
   - Also respond the markdown questions in the notebook where indicated.
3. **Documentation**:
   - Comment your code and add markdown explanations for each part of your analysis.
4. **Submission**:
   - Save your notebook and export as either PDF or HTML. If the visualizations using altair are not being shown in the html, submit a separated version with altair html. Refer to: https://altair-viz.github.io/getting_started/starting.html#publishing-your-visualization (you can use the `chart.save('chart_file.html')` method).
   - Submit to Canvas.

[5]:
```
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
import numpy as np
import scipy.stats as ss
```

1.2 Ratio and logarithm

If you use linear scale to visualize ratios, it can be quite misleading. As learned in the class, ratio values larger than 1 can vary between 1 and infinite, while ratio values smaller than 1 can vary only between 0 and 1. For instance, the ratios of 100:1 (100/1) or 1000:1 (1000/1) are represented as 100 and 1000. The corresponding distances from 1:1 (1) are 99 and 999, respectively. On the

other hand, the ratios of 1:100 (1/100) or 1:1000 (1/1000) are represented as 0.01 and 0.001. The corresponding distances from 1:1 (1) are 0.99 and 0.999, respectively. In other words, there is no symmetry between symmetric ratios!

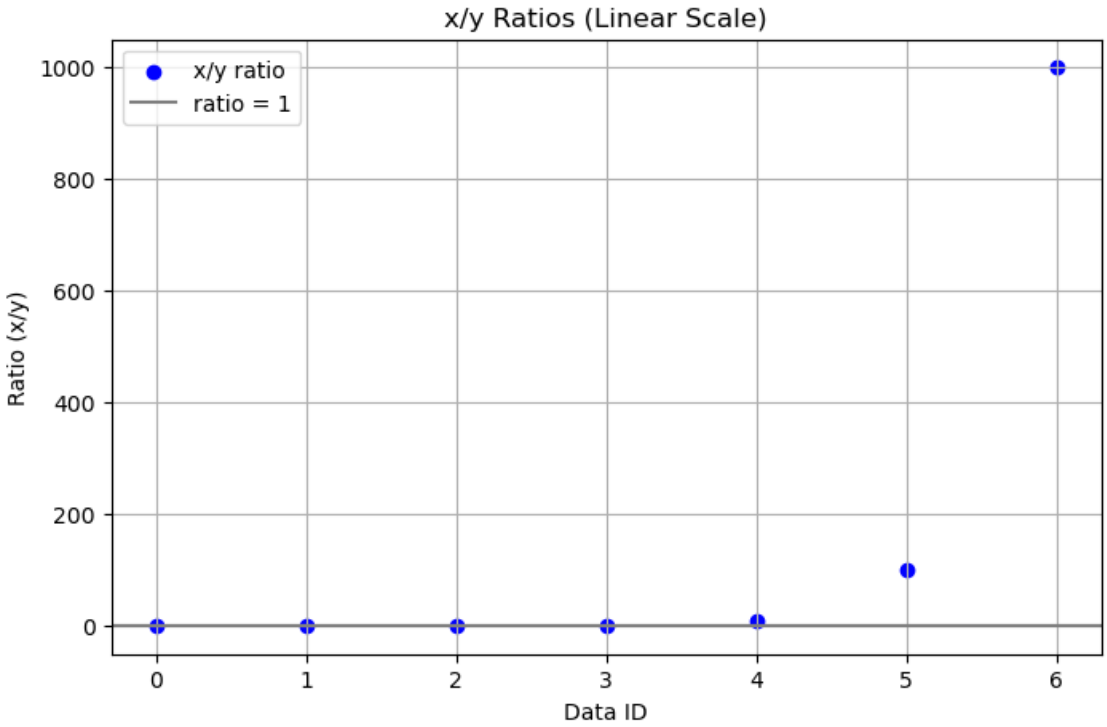
To see this clearly, let's first create some ratios.

[7]:
```
x = np.array([1,    1,   1,  1, 10, 100, 1000])
y = np.array([1000, 100, 10, 1, 1,  1,   1   ])
ratio = x/y
print(ratio)
```

```
[1.e-03 1.e-02 1.e-01 1.e+00 1.e+01 1.e+02 1.e+03]
```

Q: Plot on the linear scale using the `scatter()` function. Also draw a horizontal line at ratio=1 for a reference. The x-axis will be simply the data ID that refers to each ratio data point. Y-axis will be the ratio values.

[15]:
```
X = np.arange(len(ratio))
plt.figure(figsize=(8, 5))
plt.scatter(X, ratio, color='blue', label='x/y ratio')
plt.axhline(y=1, color='grey', label='ratio = 1')
plt.xlabel('Data ID')
plt.ylabel('Ratio (x/y)')
plt.title('x/y Ratios (Linear Scale)')
plt.legend()
plt.grid(True)
plt.show()
```



Q: Is this a good visualization of the ratio data? Why? Why not? Explain.

No, this is not a good visualization of the ratio data.

Using a linear scale distorts the interpretation—ratios greater than 1 are stretched out and dominate the chart, while ratios less than 1 are compressed near zero. This makes it hard to observe any meaningful variation in smaller values. As a result, the visualization fails to represent the data accurately or fairly.

1.2.1 Why This Happens

A linear scale treats all differences equally, regardless of their scale. But ratio data often spans several orders of magnitude. When this happens: - Large ratios appear overly prominent. - Small ratios cluster near zero. - Conceptually symmetric values like 10:1 and 1:10 appear vastly different.

This imbalance misleads the viewer about the structure of the data.

1.2.2 Better Approach

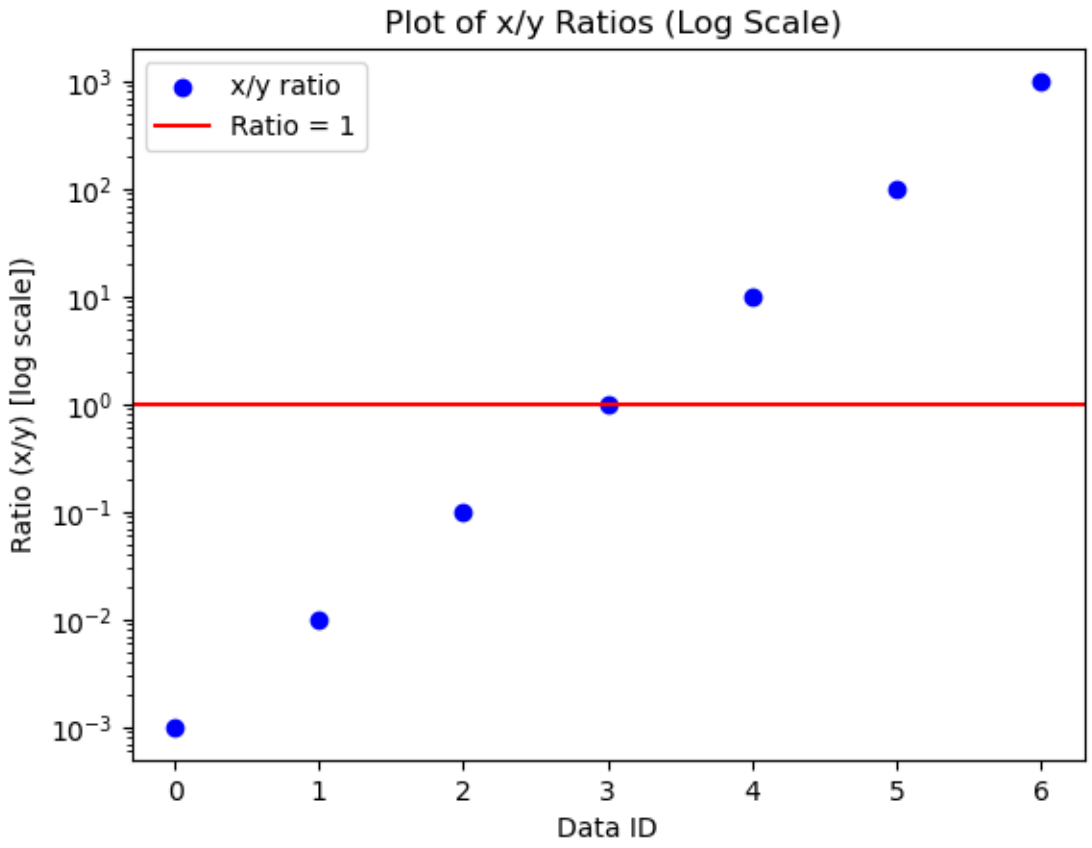
A logarithmic scale is more appropriate for visualizing ratio data. Log scales: - Spread out small values and compress large ones proportionally. - Represent multiplicative relationships clearly. - Maintain symmetry around the central ratio of 1:1 (e.g., 10:1 and 1:10 are equally distant from 1 on a log scale).

This makes the visualization clearer, more interpretable, and more faithful to the nature of the data.

2 YOUR SOLUTION HERE

Q: Can you fix it?

[20]:
```
# YOUR SOLUTION HERE
plt.scatter(X, ratio, color='blue', label='x/y ratio')
plt.axhline(y=1, color='red', label='Ratio = 1')
plt.yscale('log')
plt.title(" Plot of x/y Ratios (Log Scale)")
plt.xlabel('Data ID')
plt.ylabel('Ratio (x/y) [log scale])')
plt.legend()
plt.show()
```



2.1 Log-binning

One way to draw a histogram in log-scale, with a broadly distributed data, is by using log-binning.

Let's first see what happens if we do not use the log scale for a dataset with a heavy tail.

Q: Load the movie dataset from our Datasets folderand remove the NaN rows based on the following columns: `IMDB Rating`, `IMDB Votes`, `Worldwide Gross`, `Rotten Tomatoes Rating`.

[30]:
```
movies = pd.read_json("movies.json")

# YOUR SOLUTION HERE
mv_cols = ['IMDB Rating', 'IMDB Votes', 'Worldwide Gross', 'Rotten Tomatoes␣
 ↪Rating']
for columns in mv_cols:
    movies[columns] = pd.to_numeric(movies[columns], errors='coerce')

movies_cleaned = movies.dropna(subset=mv_cols)
movies.head()
```

[30]:
```
                        Title   US Gross  Worldwide Gross  US DVD Sales  \
0             The Land Girls   146083.0         146083.0           NaN
1     First Love, Last Rites    10876.0          10876.0           NaN
2  I Married a Strange Person   203134.0         203134.0           NaN
3       Let's Talk About Sex   373615.0         373615.0           NaN
4                       Slam  1009819.0        1087521.0           NaN

   Production Budget Release Date MPAA Rating  Running Time min Distributor  \
0          8000000.0  Jun 12 1998           R               NaN    Gramercy
1           300000.0  Aug 07 1998           R               NaN      Strand
2           250000.0  Aug 28 1998        None               NaN   Lionsgate
3           300000.0  Sep 11 1998        None               NaN   Fine Line
4          1000000.0  Oct 09 1998           R               NaN     Trimark

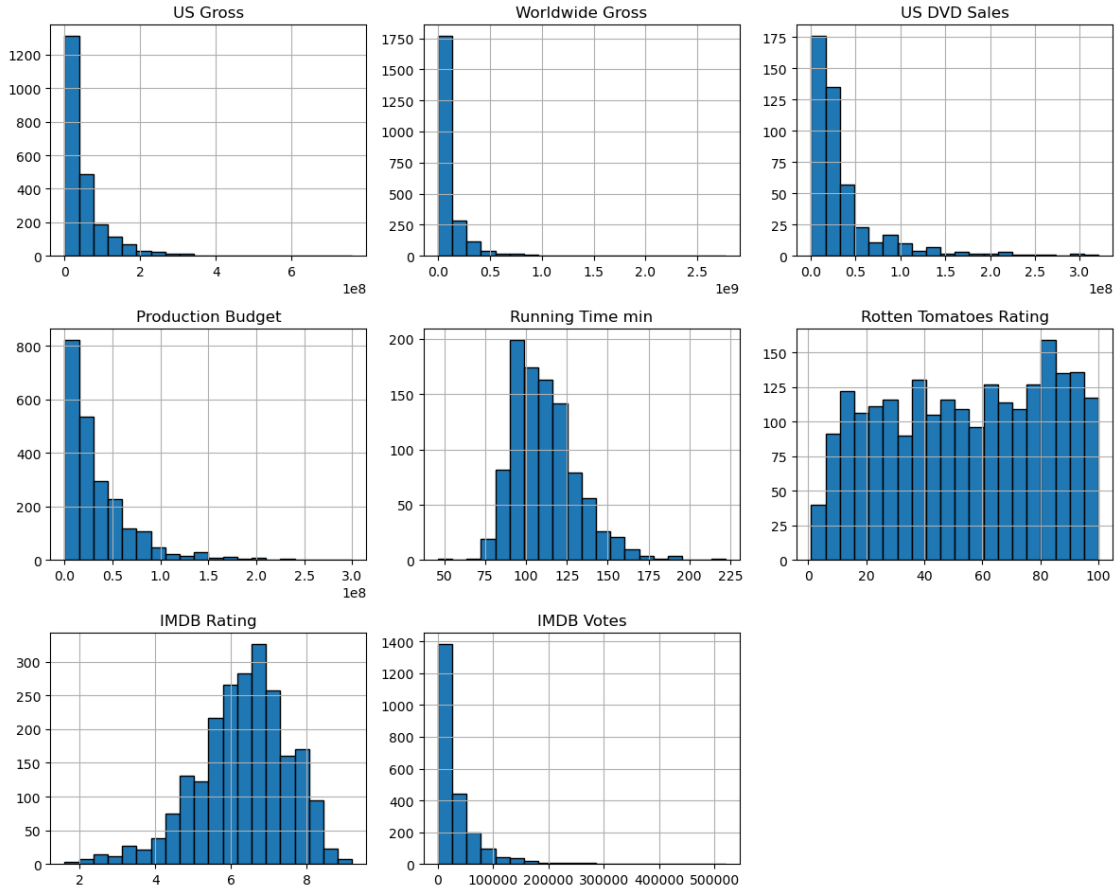
                Source Major Genre         Creative Type Director  \
0                None        None                    None     None
1                None       Drama                    None     None
2                None      Comedy                    None     None
3                None      Comedy                    None     None
4  Original Screenplay       Drama  Contemporary Fiction     None

   Rotten Tomatoes Rating  IMDB Rating  IMDB Votes
0                     NaN          6.1      1071.0
1                     NaN          6.9       207.0
2                     NaN          6.8       865.0
3                    13.0          NaN         NaN
4                    62.0          3.4       165.0
```

If you simply call `hist()` method with a dataframe object, it identifies all the numeric columns and draw a histogram for each.

Q: draw all possible histograms of the movie dataframe. Adjust the size of the plots if needed.

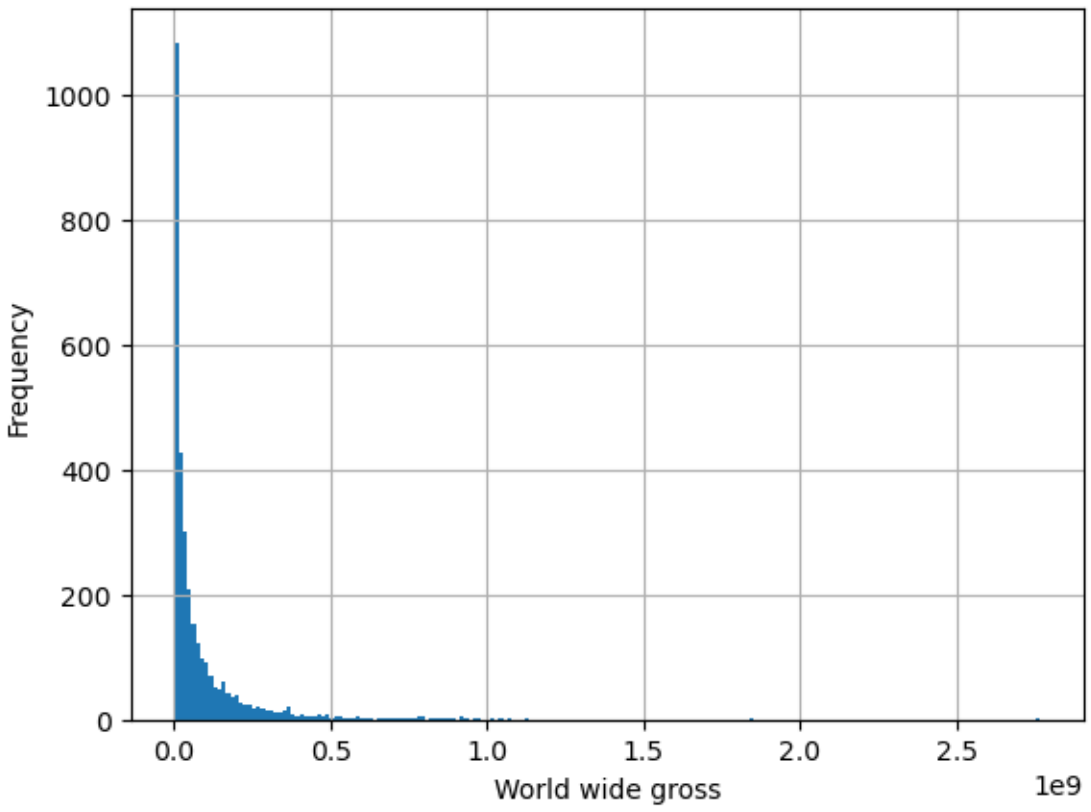
[33]:
```
# YOUR SOLUTION HERE
movies_cleaned.hist(figsize=(12, 10), bins=20, edgecolor='black')
plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()
```



As we can see, a majority of the columns are not normally distributed. In particular, if you look at the worldwide gross variable, you only see a couple of meaningful data from the histogram. Is this a problem of resolution? How about increasing the number of bins?

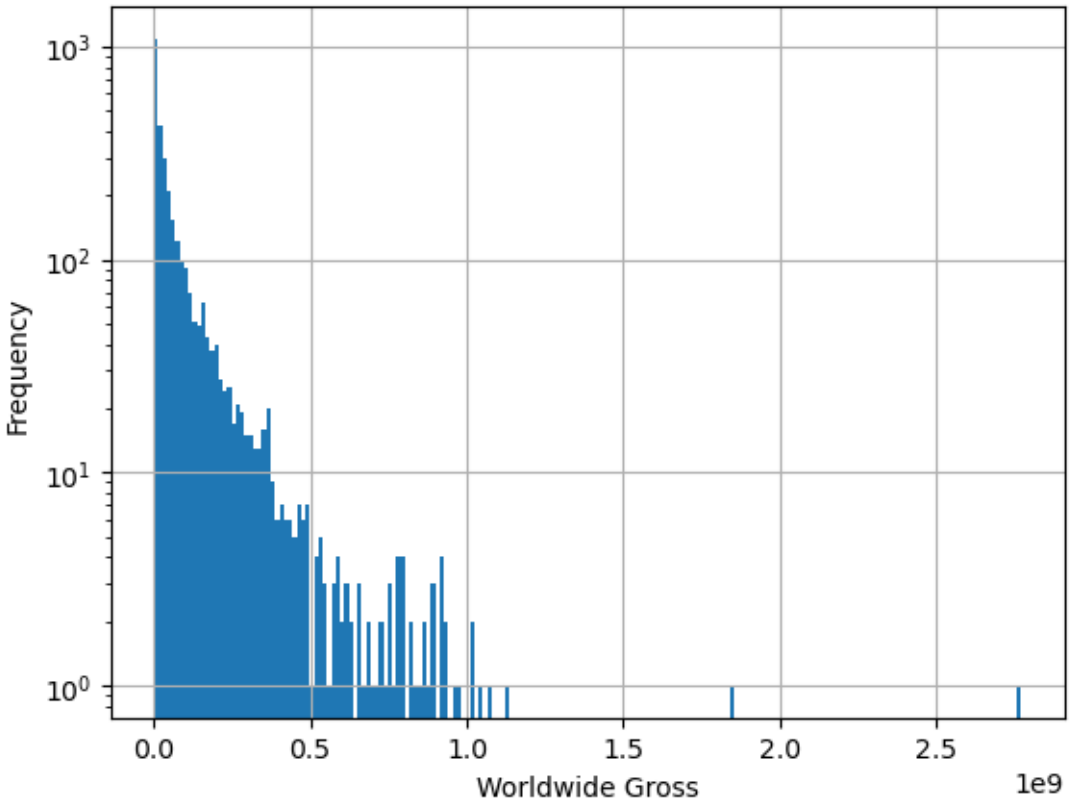
[58]:
```
ax = movies["Worldwide Gross"].hist(bins=200)
ax.set_xlabel("World wide gross")
ax.set_ylabel("Frequency")
```

[58]:
```
Text(0, 0.5, 'Frequency')
```



Maybe a bit more useful, but it doesn't tell anything about the data distribution above certain point. How about changing the vertical scale to logarithmic scale?

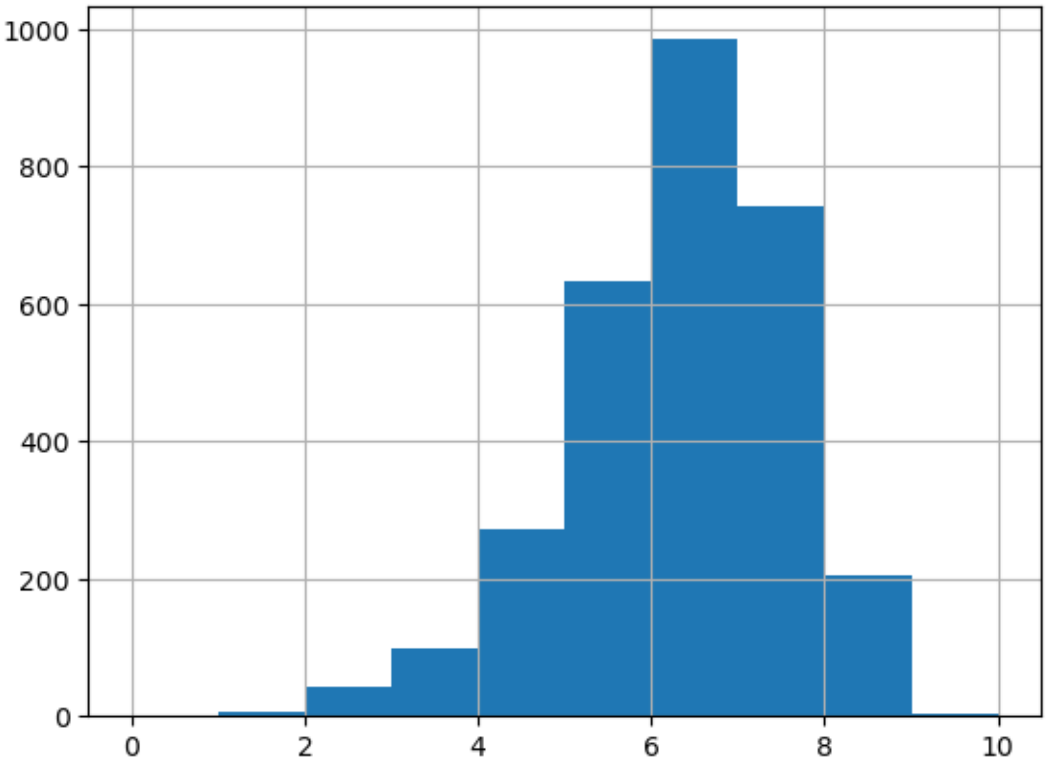
[60]:
```
# YOUR SOLUTION HERE
ax = movies["Worldwide Gross"].hist(bins=200)
ax.set_yscale('log')
ax.set_xlabel("Worldwide Gross")
ax.set_ylabel("Frequency")
plt.grid(True)
plt.show()
```



Now, let's try log-bin. Recall that when plotting histgrams we can specify the edges of bins through the `bins` parameter. For example, we can specify the edges of bins to [1, 2, 3, … , 10] as follows.

[62]:
```
movies["IMDB Rating"].hist(bins=range(0,11))
```

[62]: <Axes: >



Here, we can specify the edges of bins in a similar way. Instead of specifying on the linear scale, we do it on the log space. Some useful resources:

- Google query: python log-bin
- numpy.logspace
- numpy.linspace vs numpy.logspace

Hint: since $10^{\text{start}} = \min(\text{Worldwide Gross})$, $\text{start} = \log_{10}(\min(\text{Worldwide Gross}))$

[65]:
```
min(movies["Worldwide Gross"])
```

[65]:
```
1.0
```

Because there seems to be movie(s) that made $0, and because log(0) is undefined & log(1) = 0, let's add 1 to the variable.

[67]:
```
movies["Worldwide Gross"] = movies["Worldwide Gross"]+1.0
```

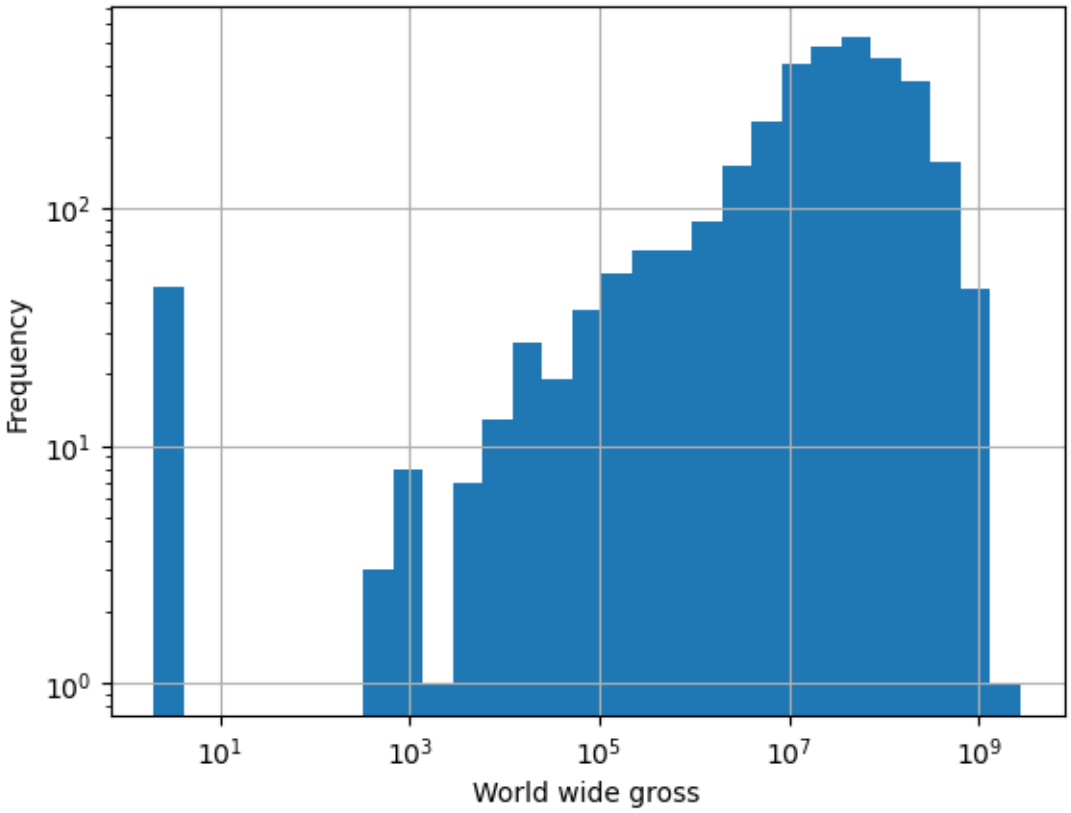
[68]:
```
# YOUR SOLUTION HERE
start = np.log10(movies["Worldwide Gross"].min())
end = np.log10(movies["Worldwide Gross"].max())
bins = np.logspace(start, end, num=30)
print(bins)
```

```
[2.00000000e+00 4.13276773e+00 8.53988456e+00 1.76466797e+01
 3.64648142e+01 7.53503037e+01 1.55702652e+02 3.21741448e+02
 6.64841337e+02 1.37381741e+03 2.83883413e+03 5.86612105e+03
 1.21216579e+04 2.50479983e+04 5.17587796e+04 1.06953507e+05
 2.21007001e+05 4.56685302e+05 9.43687140e+05 1.95001988e+06
 4.02948962e+06 8.32647234e+06 1.72056881e+07 3.55535563e+07
 7.34672951e+07 1.51811633e+08 3.13701110e+08 6.48226912e+08
 1.33948563e+09 2.76789150e+09]
```

Now we can plot a histgram with log-bin. Set both axis to be log-scale.

[75]:
```
ax = (movies["Worldwide Gross"]+1.0).hist(bins=bins)
ax.set_yscale('log')
ax.set_xscale('log')
ax.set_xlabel("World wide gross")
ax.set_ylabel("Frequency")
```

[75]:
```
Text(0, 0.5, 'Frequency')
```



What is going on? Is this the right plot?

Q: explain and fix

This plot shows a histogram of worldwide gross using a logarithmic x-axis. However, it is **not quite right**.

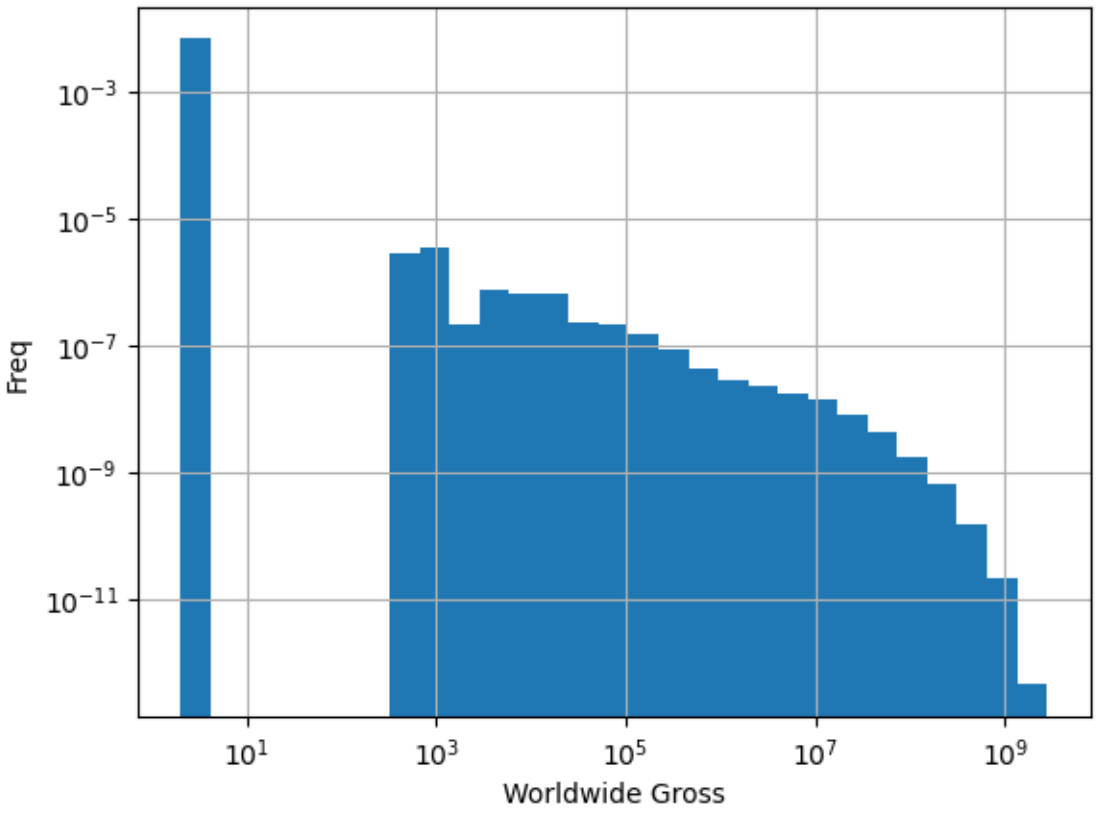
The issue is that while the x-axis is set to log scale, the histogram **bins are still spaced linearly**, not logarithmically. This leads to misleading bar widths and uneven data grouping. You can see large gaps in low-value regions and dense stacking in high-value regions.

2.1.1 Fix

To properly visualize data on a logarithmic x-axis, the bins themselves must be spaced **logarithmically**, not just the axis. Otherwise, the counts in wide bins will overpower those in narrow ones, leading to a distorted view.

[78]:
```
# YOUR SOLUTION HERE
ax = movies["Worldwide Gross"].hist(bins=bins, density=True)
ax.set_xscale('log')
ax.set_yscale('log')
ax.set_xlabel("Worldwide Gross")
ax.set_ylabel("Freq")
```

[78]:
```
Text(0, 0.5, 'Freq')
```



2.2 CCDF

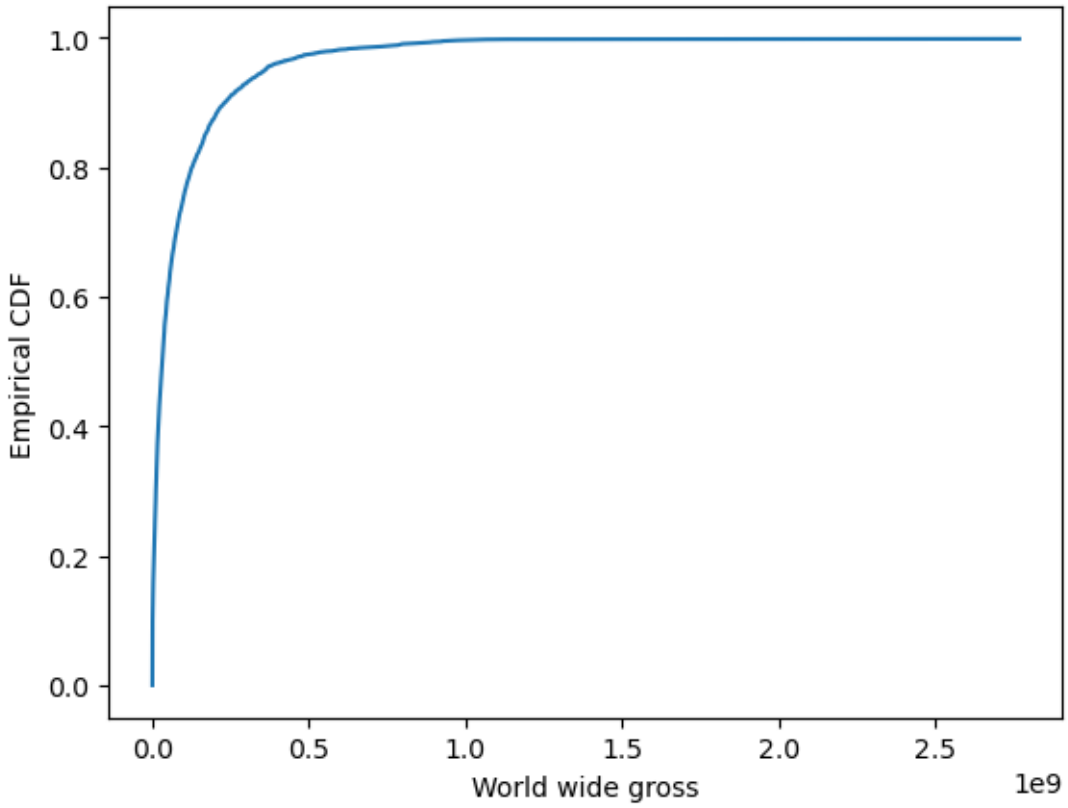
The cumulative distribution function $F_X(x)$ at x is defined by

$$F_X(x) = P(X \leq x),$$

which is, in other words, the probability that X takes a value less than or equal to x. When empirically calculated (empirical CDF), $F_X(x)$ is the fraction of data points that are less than or equal to x. CDF allows us to examine any percentile of the data distribution and is also useful for comparing distributions.

However, when the data spans multiple orders of magnitude, CDF may not be useful. Let's try.

[81]:
```
gross_sorted = movies["Worldwide Gross"].sort_values()
N = len(gross_sorted)
Y = np.linspace(1/N, 1, num=N)
plt.xlabel("World wide gross")
plt.ylabel("Empirical CDF")
_ = plt.plot(gross_sorted, Y)
```



Although the movies that are interesting are those with large worldwide gross, we don't see any details about their distribution as they are all close to 1. In other words, CDF sucks at revealing the details of the tail.

CCDF is a nice alternative to examine distributions with heavy tails. The idea is same as CDF, but the direction of aggregation is opposite. Because we are starting from the largest value, it can reveal the details of those large values (tail).

CCDF is defined as follows:

$$\bar{F}_X(x) = P(X > x)$$

And thus,

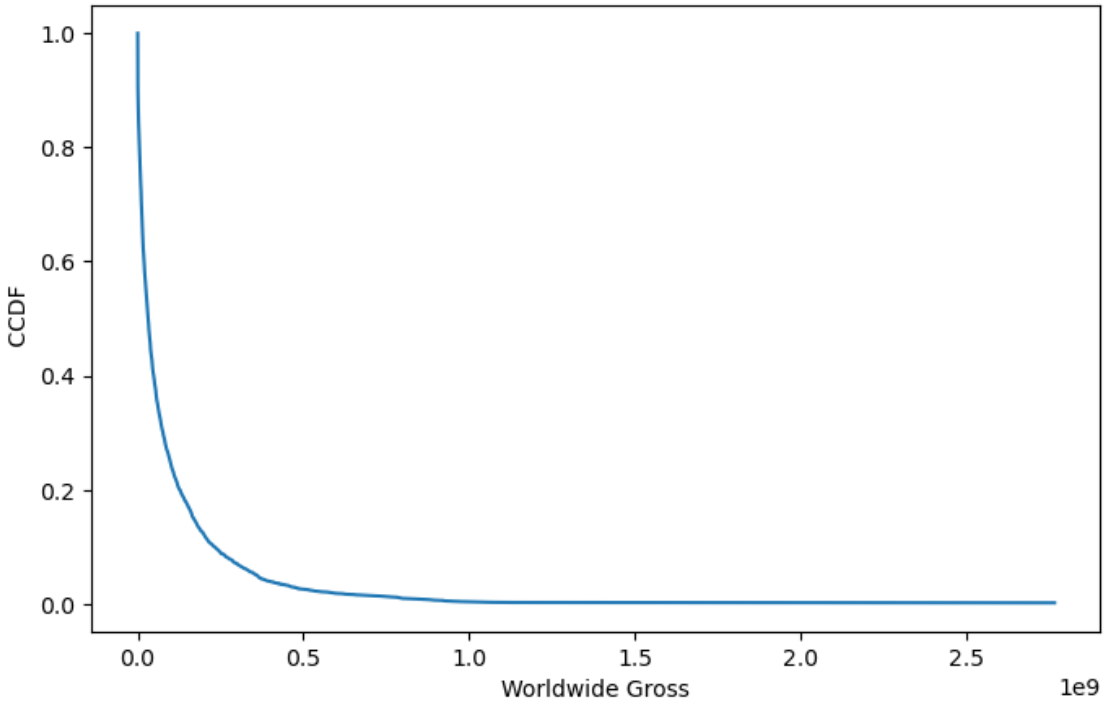
$$\bar{F}_X(x) = P(X > x) = 1 - F_X(x)$$

In other words, we can use CDF to calculate CCDF.

Q: draw CCDF using the CDF code above.

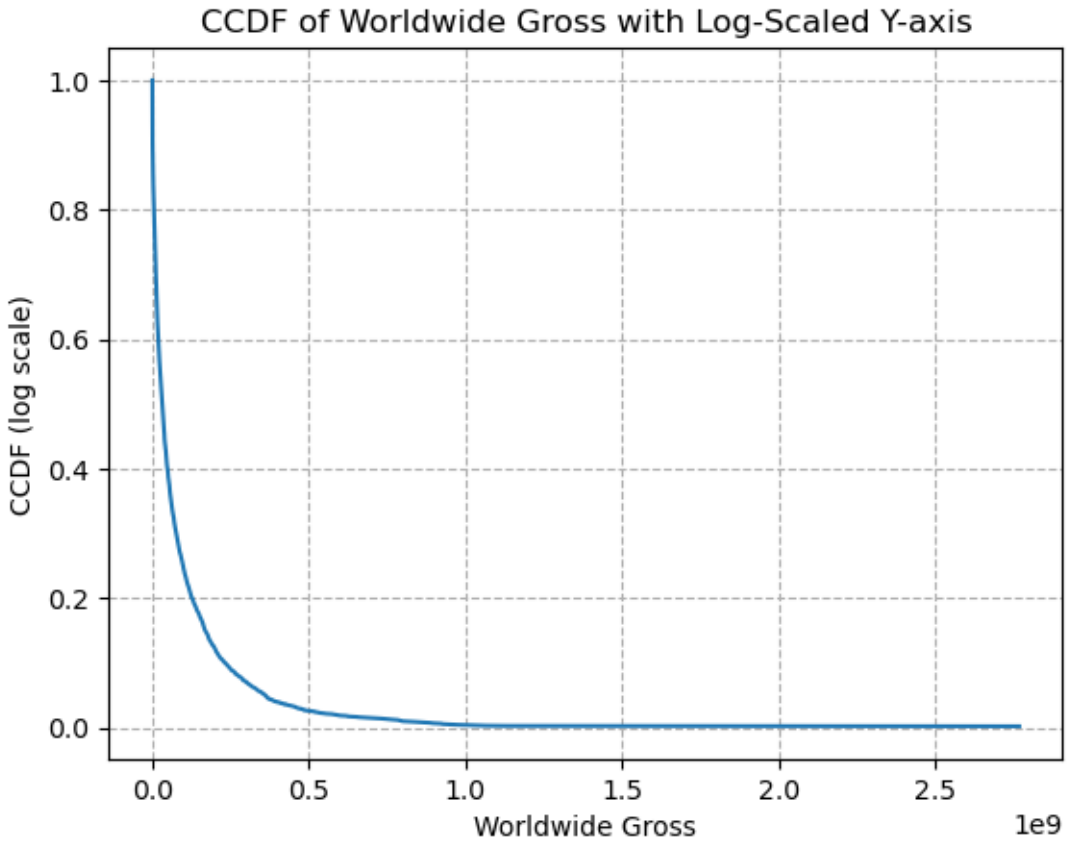
[119]:
```
# YOUR SOLUTION HERE
ccdf = 1 - Y
plt.figure(figsize=(8, 5))
plt.xlabel("Worldwide Gross")
plt.ylabel(" CCDF")
plt.plot(gross_sorted, ccdf)
```

[119]: [<matplotlib.lines.Line2D at 0x2b9f9c1cdd0>]



How about making the y axis in log scale?

[122]:
```
# YOUR SOLUTION HERE
plt.plot(gross_sorted, ccdf)
plt.xlabel("Worldwide Gross")
plt.ylabel("CCDF (log scale)")
plt.title("CCDF of Worldwide Gross with Log-Scaled Y-axis")
plt.grid(True, which='both', ls='--')
plt.show()
```



Although this is technically the correct CCDF plot, there is a very subtle issue. Do you see the vertical line at the rightmost side of the CCDF plot? To understand what's going on, let's look at the Y values of this plot. We used 1 - CDF to calculate CCDF. So,

[125]:
```
1 - Y
```

[125]:
```
array([9.99687598e-01, 9.99375195e-01, 9.99062793e-01, …,
       6.24804749e-04, 3.12402374e-04, 0.00000000e+00])
```

What happens when we take the log of these values?

[128]:
```
np.log(1-Y)
```

```
C:\Users\aryan\AppData\Local\Temp\ipykernel_31172\1767632406.py:1:
RuntimeWarning: divide by zero encountered in log
  np.log(1-Y)
```

[128]:
```
array([-3.12451182e-04, -6.25000020e-04, -9.37646576e-04, …,
       -7.37807136e+00, -8.07121854e+00,            -inf])
```

Because the last value of 1 - Y is 0.0, we got `-inf` as the log value. That means, the largest value's (let's say x) coordinate in our CCDF plot will be $(x, -inf)$ if we use a log scale for our y-axis. And thus we will not be able to see it in the plot. This occurs because we are drawing CDF in a simplified way. In reality, ECDF and ECCDF are step functions and this shouldn't matter. However, because we are drawing a line between the points, we are getting this issue.

This is somewhat problematic because the largest value in our dataset can be quite important and therefore we want to see it in the plot!

This is why, in practice, we sometimes use "incorrect" version of CCDF. We can consider $\bar{F}_X(x)$ as a "flipped" version of CDF.

$$\bar{F}_X(x) = P(X \geq x)$$

instead of

$$\bar{F}_X(x) = P(X > x)$$

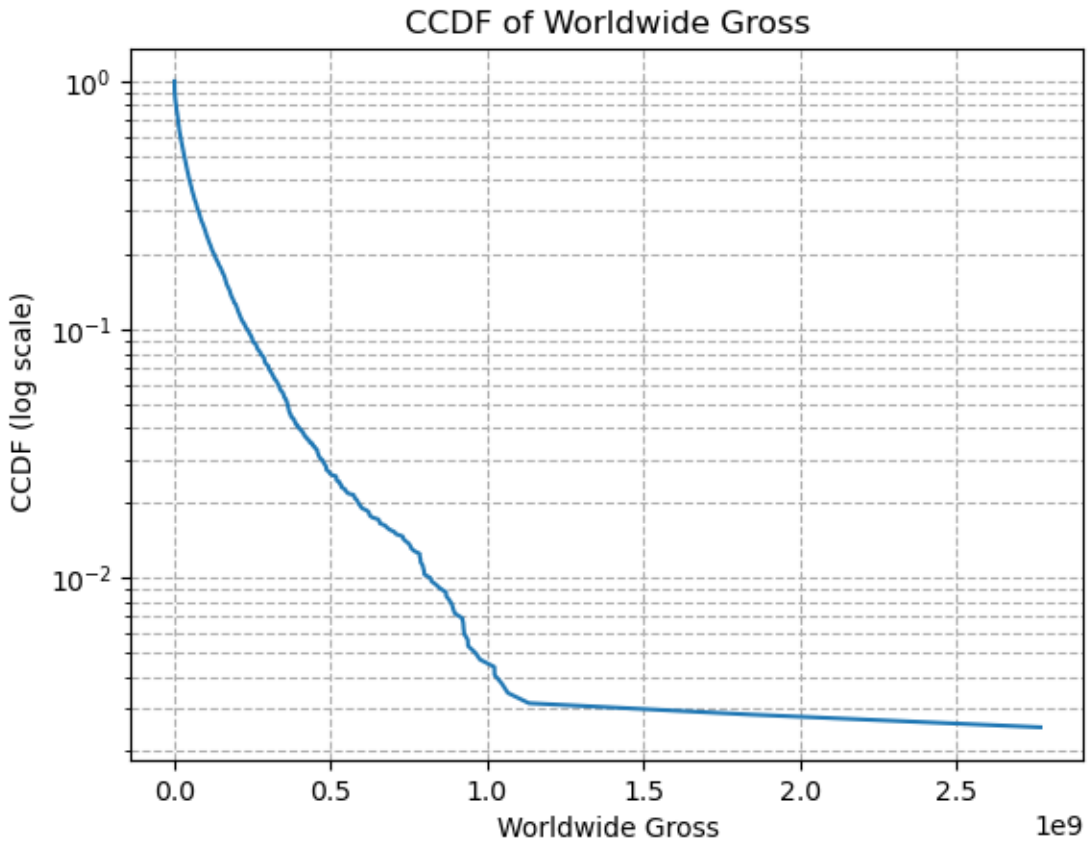
In doing so, we can see the largest value in the data in our CCDF plot. We can also draw the correct version of CCDF, but this quick-and-dirty version is often easier and good enough to show what we want to show.

A simple way is just to define the y coordinates as follows:

[131]:
```
Y = np.linspace( 1.0, 1/N, num=N)
```

Q: Draw a CCDF of worldwide gross data. Use log scale for y-axis.

[134]:
```
plt.plot(gross_sorted, Y)
plt.yscale('log')
plt.xlabel("Worldwide Gross")
plt.ylabel(" CCDF (log scale)")
plt.title("CCDF of Worldwide Gross")
plt.grid(True, which='both', ls='--')
plt.show()
```



A straight line in semilog scale means exponential decay (cf. a straight line in log-log scale means power-law decay). So it seems like the amount of money a movie makes across the world follows *roughly* an exponential distribution, while there are some outliers that make insane amount of money.

Q: Which is the most successful movie in our dataset?

AVATAR

You can use the following

- `idxmax()`: https://pandas.pydata.org/pandas-docs/stable/generated/pandas.DataFrame.idxmax.html
- `loc`: https://pandas.pydata.org/pandas-docs/stable/generated/pandas.DataFrame.loc.html

[146]:
```
# YOUR SOLUTION HERE
movie_index = movies["Worldwide Gross"].idxmax()
most_successful_movie = movies.loc[movie_index]
most_successful_movie
```

[146]:
```
Title                              Avatar
US Gross                      760167650.0
Worldwide Gross              2767891501.0
```

```
US DVD Sales                        146153933.0
Production Budget                   237000000.0
Release Date                        Dec 18 2009
MPAA Rating                               PG-13
Running Time min                            NaN
Distributor                     20th Century Fox
Source                       Original Screenplay
Major Genre                               Action
Creative Type                  Science Fiction
Director                         James Cameron
Rotten Tomatoes Rating                    83.0
IMDB Rating                                8.3
IMDB Votes                            261439.0
Name: 1234, dtype: object
```

2.3 Part 2: Ukraine-Russia Conflict Tweets

Now let's see some social media data. Tweets about the Ukraine-Russia conflict.

We load the dataset, define data types for specific columns, and display a sample of the data. The analysis will include Visualizing the distribution of the 'RetweetCount' field.

[149]:
```
dtypes = {
    "Tweet Id" : "str",
    "ConversationId": "str",
}
df_ukraine = pd.read_csv('Ukraine_tweets.csv',dtype=dtypes)
df_ukraine.head()
```

```
C:\Users\aryan\AppData\Local\Temp\ipykernel_31172\3331735757.py:5: DtypeWarning:
Columns (0,2,3,4,5,6,7,11,13,14,15,16,17,18) have mixed types. Specify dtype
option on import or set low_memory=False.
  df_ukraine = pd.read_csv('Ukraine_tweets.csv',dtype=dtypes)
```

[149]:

	Datetime	Tweet Id	Text	Username
0	2022-02-24 03:12:47+00:00	1496684505247141897	Massive explosions rocks #Kharkiv. #Russia …	IdeologyWars
1	2022-02-24 03:18:54+00:00	1496686044275695616	Kharkiv is officially being struck by major…	IdeologyWars
2	2022-02-24 03:22:42+00:00	1496687000375726080	More angles on that strike in Kharkiv. #Rus…	IdeologyWars
3	2022-02-24 03:25:36+00:00	1496687731434565636	BM-21 Grad strikes opening on #Mariupol cit…	IdeologyWars
4	2022-02-24 03:27:28+00:00	1496688201242759177	Damage caused by strike in Kharkiv… #Russ…	IdeologyWars

```
                                        Permalink  \
0  https://twitter.com/IdeologyWars/status/149668…
1  https://twitter.com/IdeologyWars/status/149668…
2  https://twitter.com/IdeologyWars/status/149668…
3  https://twitter.com/IdeologyWars/status/149668…
4  https://twitter.com/IdeologyWars/status/149668…

                              User Outlinks CountLinks  ReplyCount  \
0  https://twitter.com/IdeologyWars      NaN        NaN         2.0
1  https://twitter.com/IdeologyWars      NaN        NaN         2.0
2  https://twitter.com/IdeologyWars      NaN        NaN         1.0
3  https://twitter.com/IdeologyWars      NaN        NaN        21.0
4  https://twitter.com/IdeologyWars      NaN        NaN         7.0

   RetweetCount  LikeCount QuoteCount       ConversationId Language  \
0          17.0       47.0        1.0  1496493071495987201       en
1         104.0      188.0        8.0  1496493071495987201       en
2          41.0       75.0        5.0  1496493071495987201       en
3         407.0     1099.0      149.0  1496493071495987201       en
4         152.0      207.0       22.0  1496493071495987201       en

                                           Source  \
0  <a href="https://mobile.twitter.com" rel="nofo…
1  <a href="https://mobile.twitter.com" rel="nofo…
2  <a href="https://mobile.twitter.com" rel="nofo…
3  <a href="https://mobile.twitter.com" rel="nofo…
4  <a href="https://mobile.twitter.com" rel="nofo…

                                            Media QuotedTweet  \
0  [Photo(previewUrl='https://pbs.twimg.com/media…         NaN
1  [Video(thumbnailUrl='https://pbs.twimg.com/ext…         NaN
2  [Video(thumbnailUrl='https://pbs.twimg.com/ext…         NaN
3  [Video(thumbnailUrl='https://pbs.twimg.com/ext…         NaN
4  [Video(thumbnailUrl='https://pbs.twimg.com/ext…         NaN

  MentionedUsers                                          hashtag  \
0            NaN  ['#Kharkiv.', '#Russia', '#Ukraine', '#Ukraine…
1            NaN  ['#Russia', '#Ukraine', '#UkraineWar', '#Russi…
2            NaN  ['#Russia', '#Ukraine', '#UkraineWar', '#Russi…
3            NaN  ['#Mariupol', '#Russia', '#Ukraine', '#Ukraine…
4            NaN  ['#Russia', '#Ukraine', '#UkraineWar', '#Russi…

   hastag_counts
0            9.0
1            8.0
2            8.0
3            9.0
```

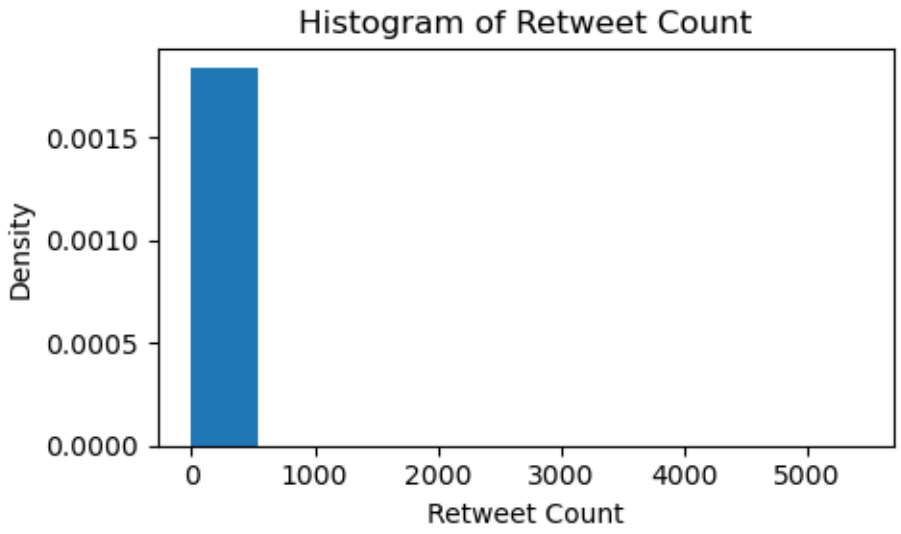
```
4              8.0
```

Q: Why do you need to define data types for specific columns?

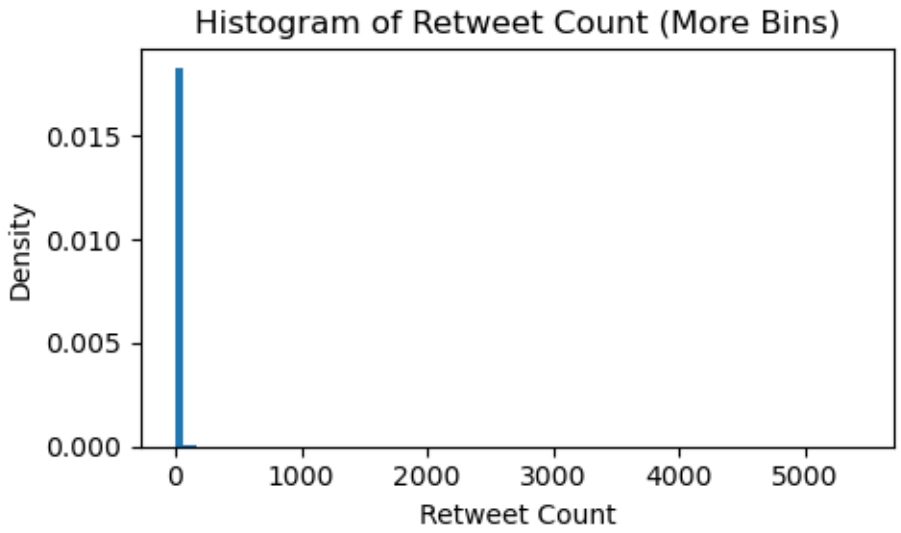
Certain columns like `Tweet Id` and `ConversationId` contain large numerical values that can exceed the default integer limits or lose precision when stored as floats. By explicitly setting them as strings, we preserve their exact values and avoid potential issues with formatting, rounding, or misinterpretation during analysis.

Let's visualize the distribution of the number of times a tweet was retweeted.

[152]:
```
# distribution of number of retweets
plt.figure(figsize=(5, 3))
retweetCount = df_ukraine['RetweetCount'].dropna()
plt.hist(retweetCount, density=True)
plt.xlabel('Retweet Count')
plt.ylabel('Density')
plt.title('Histogram of Retweet Count')
plt.tight_layout()
plt.show()
```



[153]:
```
# increase number of bins
plt.figure(figsize=(5, 3))
retweetCount = df_ukraine['RetweetCount'].dropna()
plt.hist(retweetCount, bins=100, density=True)
plt.xlabel('Retweet Count')
plt.ylabel('Density')
plt.title('Histogram of Retweet Count (More Bins)')
plt.tight_layout()
plt.show()
```



2.3.1 Q: What is the problem with this plot?

The data is highly skewed—most retweet counts are small, but a few are very large. On a linear scale, all the small values get crammed into one bin, making the histogram unreadable.

2.3.2 Fix

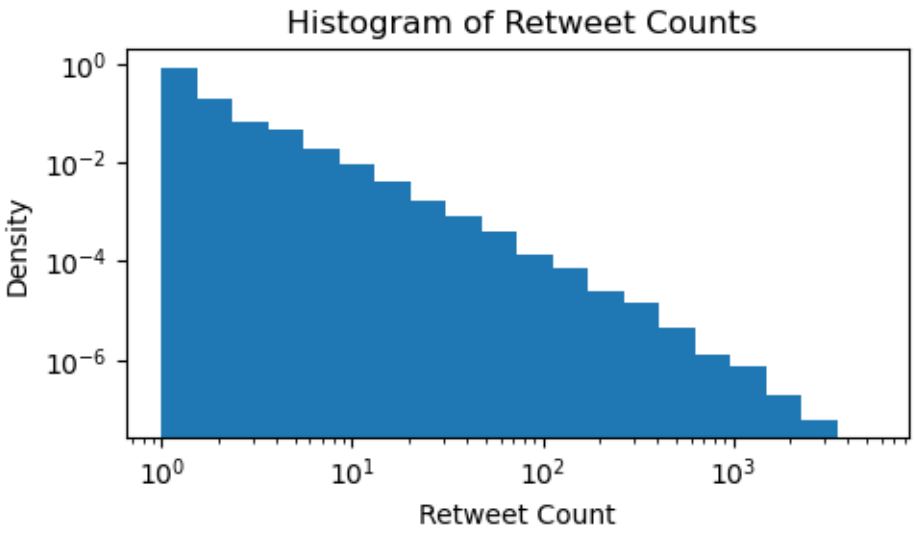
Use a **logarithmic scale** for the x-axis to better spread out the values and reveal the distribution.

Q: What is the problem with this plot?

Let's try to use a log-log scale to visualize the distribution of retweet counts.

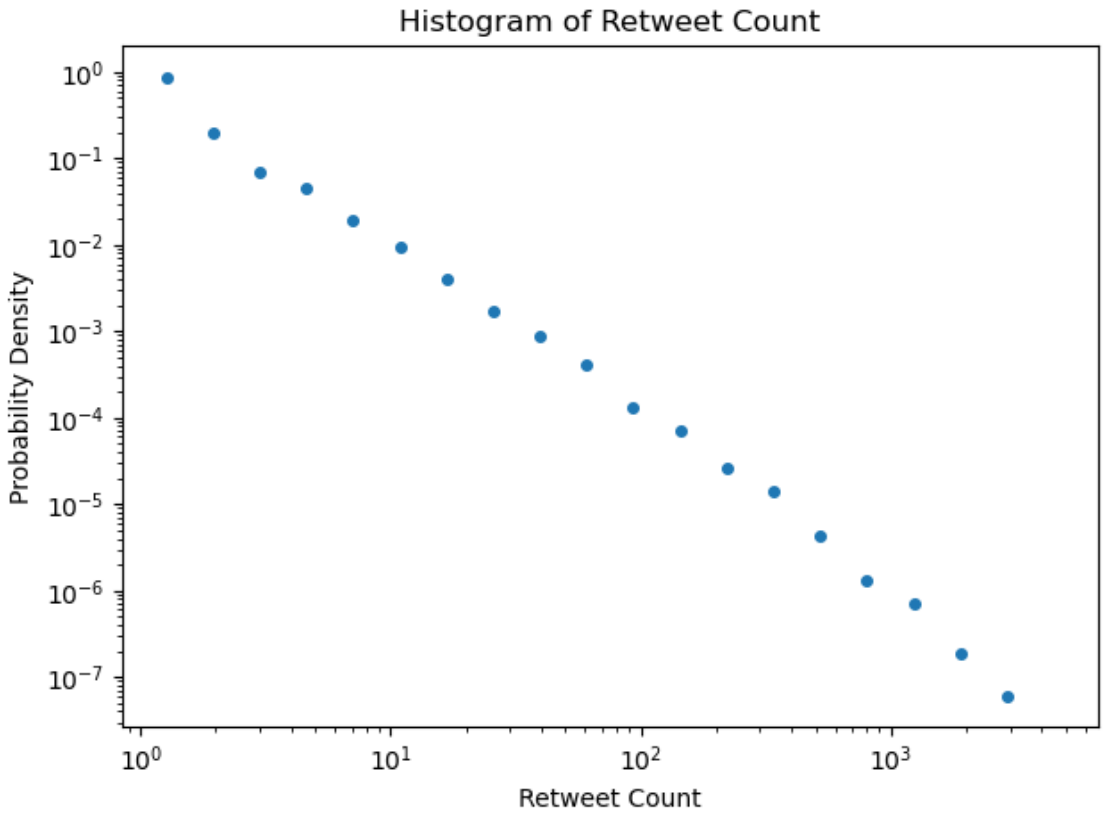
[157]:
```
# distribution of number of retweets
plt.figure(figsize=(5, 3))
retweetCount = df_ukraine['RetweetCount'].dropna()
retweetCount = retweetCount[retweetCount > 0]
bins = np.logspace(np.log10(retweetCount.min()), np.log10(retweetCount.max()),␣
↪21)
plt.hist(retweetCount, density=True, bins=bins)

plt.xscale('log')
plt.yscale('log')
plt.xlabel('Retweet Count')
plt.ylabel('Density')
plt.title(' Histogram of Retweet Counts')
plt.tight_layout()
plt.show()
```



We can also use a circles instead of bars. This way, we can see the distribution more clearly.

[165]:
```
bins = np.logspace(np.log10(retweetCount.min()), np.log10(retweetCount.max()),␣
↪21)
hist, bins = np.histogram(retweetCount, bins=bins, density=True)
centerBins = 0.5 * (bins[1:] + bins[:-1])
plt.plot(centerBins, hist, "o ", ms=4)
plt.xscale('log')
plt.yscale('log')
plt.xlabel('Retweet Count')
plt.ylabel('Probability Density')
plt.title('Histogram of Retweet Count')
plt.tight_layout()
plt.show()
```



Q: What do you see now?

The distribution follows a straight line on a log-log scale, indicating a heavy-tailed pattern.

Q: Can you guess what this distribution looks like?

This looks like a power-law distribution.

Q: What are the characteristics of this distribution type?

- Few values are very large.
- Most values are very small.
- Follows a straight line on log-log plots.

Q: Can you guess why this distribution type may appear in social media metrics such as retweet count?

A small number of posts go viral, while most get few retweets.

Q: Can you give examples where it may appear?

- Follower counts
- Video views
- Website traffic
- City populations

[ ]: